

Page 1 — NLP Fundamentals

Knowledge Base Document: Nlp | Page 1 of 5

Natural Language Processing, or NLP, applies computational methods to human language. Tasks include text classification, sentiment analysis, named entity recognition, information extraction, question answering, summarization, and machine translation.

Text is unstructured and ambiguous. The same word can have multiple meanings, spelling variations can represent the same concept, and context can change interpretation. NLP systems therefore require representations that capture useful linguistic information.

Tokenization divides text into units such as words, subwords, or characters. Modern transformer systems commonly use subword tokenization because it handles unknown words and vocabulary size efficiently.

Normalization may include case conversion, whitespace cleanup, Unicode normalization, and removal or transformation of unwanted characters. Aggressive normalization can destroy information, so preprocessing should match the task.

Stop-word removal and stemming were common in traditional NLP pipelines, but they are not universally beneficial. Modern transformer models often operate on raw or lightly normalized text because contextual representations can use function words.

Page 2 — Classical NLP Representations

Knowledge Base Document: Nlp | Page 2 of 5

Bag-of-Words represents a document using counts of vocabulary terms. It is simple and interpretable but ignores word order and semantic similarity.

TF-IDF weights terms according to their frequency in a document and rarity across a corpus. It often provides a strong baseline for document classification and information retrieval.

N-grams capture sequences of adjacent tokens, allowing simple models to represent some local word order. Increasing n can greatly increase dimensionality, so vocabulary controls and regularization may be necessary.

Word embeddings represent tokens as dense vectors. Word2Vec and GloVe learn representations in which words used in similar contexts tend to have related vectors.

Classical pipelines often combine TF-IDF or embeddings with logistic regression, linear SVM, naive Bayes, or tree-based methods. They remain useful because they are fast, interpretable, and inexpensive.

Page 3 — Transformers and Modern NLP

Knowledge Base Document: Nlp | Page 3 of 5

Transformers use attention mechanisms to model relationships between tokens. Self-attention allows each token representation to incorporate information from other tokens in the sequence.

BERT introduced bidirectional transformer pretraining for language understanding tasks. Models based on the BERT architecture can be fine-tuned for classification, token classification, and extractive question answering.

Generative transformer models predict or generate sequences and can perform tasks such as summarization, dialogue, code generation, and question answering. Large language models are trained on very large corpora and can be adapted through prompting or additional training.

Attention has computational costs that grow with sequence length in standard transformer architectures. Long-context systems therefore use engineering techniques such as efficient attention, chunking, retrieval, or specialized architectures.

Token limits matter in NLP applications. Long documents should often be split into meaningful chunks, and retrieval systems should return only the context relevant to the current query.

Page 4 — NLP Applications and Evaluation

Knowledge Base Document: Nlp | Page 4 of 5

Text classification assigns labels to documents or messages. A standard workflow includes collecting labeled data, splitting it appropriately, preprocessing, vectorization or tokenization, model training, evaluation, and error analysis.

Named Entity Recognition identifies spans corresponding to entities such as people, organizations, locations, dates, and products. Entity extraction is useful in document processing, search, and knowledge graph construction.

Information retrieval ranks documents or passages according to their relevance to a query. Lexical methods such as BM25 are strong baselines, while dense retrieval uses embedding vectors and similarity search.

NLP evaluation depends on the task. Classification commonly uses precision, recall, and F1. Sequence generation may use ROUGE or BLEU-like metrics, but automatic metrics do not always capture factuality, relevance, or fluency.

Human evaluation and error analysis remain important. A model can achieve good aggregate metrics while failing systematically on rare entities, long inputs, negation, domain terminology, or ambiguous language.

Page 5 — NLP in RAG Systems

Knowledge Base Document: Nlp | Page 5 of 5

Retrieval-Augmented Generation combines document retrieval with language generation. Documents are loaded, cleaned, divided into chunks, embedded into vectors, and stored in a vector index. A user query is embedded and used to retrieve relevant chunks.

Chunk size affects retrieval quality. Very small chunks may lose context, while very large chunks may dilute relevance and exceed the model's useful context budget. Overlap can preserve information across chunk boundaries.

Metadata such as source filename, page number, section title, and document type should be stored with each chunk. Metadata improves traceability and allows filtering before or after similarity search.

A RAG answer should be grounded in retrieved evidence. The application can instruct the generation model to cite or identify sources and to say when the retrieved context does not contain enough information.

Common RAG failure modes include poor chunking, irrelevant retrieval, duplicate passages, stale indexes, prompt injection in documents, and hallucinated answers. Evaluation should separately measure retrieval quality and answer quality.